QuickSyn SCPI Controller and Telemetry Logger

William W. Wallace, NRAO/GBO

Version 0.0.6

Contents

1. Files **1**

2. Link parameters **3**

3. SCPI coverage **3**
3.1 Mandatory settling delays . . . 4
3.2 Interlocks enforced . . . 5

4. The 64-byte buffer, and why frequencies are encoded two ways **5**

5. Logging **5**
5.1 Sample rate . . . 6
5.2 Memory discipline . . . 6
5.3 Parallelism and GPU . . . 7
5.4 Live SCPI console . . . 7

6. Veusz documents, reduction and plots **9**
6.1 Why `.vszh5` and not `.vsz` . . . 9
6.2 The writer: `vszh5_writer.py` (from `isp_speed_monitor.py`) . . . 10
6.3 What the session document contains . . . 11
6.4 From the controller . . . 12
6.5 Rendering, and Veusz command-line limitations . . . 12
6.6 Statistics . . . 13
6.7 PDF bookmarks and the LuaLaTeX session report . . . 13

7. Model capability table **14**

8. Memory hygiene **15**

9. Documented conflicts and how they are handled **15**

10. Verification status **16**

11. Revision history **19**

12. References **24**
Sourcing confidence . . . 26

13. Open items **28**

14. Licence **28**
14.1 Third-party components . . . 29

Version 0.0.6 · Python 3.12 · Qt.py → PySide6 · 2026-10-11

Author	William W. Wallace
Company	NRAO/GBO
Email	wwallace@nrao.edu

Licence	GPL-3.0-or-later — see `LICENSE` and §14
Versioning	Semantic Versioning 2.0.0 [15]; `0.y.z` means initial development
Status	Internal draft — **not yet verified against hardware**

SCPI control of the NI / Phase Matrix QuickSyn® microwave frequency synthesiser family over the USB virtual COM port, with a timestamped log of every command generated and a telemetry stream sampled at a user-selected rate.

---

1. Files

File	Version	Purpose
`quicksyn_scpi_gui_v0_0_6.py`	0.0.6	Controller, logger and GUI with the live SCPI console (single file, Spyder `# %%` cells)
`quicksyn_veusz_export_v0_0_6.py`	0.0.6	Reduction and statistics; `export_veusz()` writes the session `.vszh5`; renders PDF (with bookmarks) and PNG from it; writes and compiles the LuaLaTeX session report
`vszh5_writer.py`	0.0.6	**Veusz HDF5 writer** (`VeuszWriter`, `VszDoc`, `vz_text`, …) from `isp_speed_monitor.py`: h5py only, no Veusz needed to write; `render_pages()` and `add_pdf_bookmarks()`; reusable in other programs
`environment_quicksyn.yml`	0.0.6	Mamba environment definition
`README_quicksyn_controller.md`	0.0.6	This file
`README_quicksyn_controller.pdf` / `.tex`	0.0.6	This file typeset with LuaLaTeX (bookmark hierarchy of the sections, outline pane opens); the `.tex` is the editable LaTeX source
`LICENSE`	—	Full GNU GPL version 3 text, unmodified
`tests/check_release_metadata.py`	0.0.6	Headers, SemVer, SPDX, history entries, `LICENSE` hash, `--version`, About dialog, every file at one version
`tests/check_smoke.py`	0.0.6	Driver, simulator, logging, sampler and full-GUI checks, including the controller's own `.vszh5` export
`tests/check_reduction.py`	0.0.6	Rate, jitter, ADEV and moving-statistics maths

File	Version	Purpose
`tests/check_vszh5.py`	0.0.6	Writer and `export_veusz()`: layout, types, tags, escaping, DST, bit-exact data, pages; render plumbing (Windows paths, broken or unstartable Veusz, fallback); PDF page scale, bookmarks, re-render; LuaLaTeX report; real-Veusz load, re-save comparison and renders when Veusz is available
`tests/check_console.py`	0.0.6	The live SCPI console: line format and alignment, thread safety, a simulated session against `transactions.csv`, a 200 000-exchange pause (memory and Resume time), filter, clear, save
`tests/run_all.py`	0.0.6	Runs the five checks in parallel and prints a summary
`upstream/veusz-4.2.1-vszh5-load-quit-pdfdpi.patch`	—	Patch for Veusz itself (not needed by this package): `.vszh5` in `--export` and `Load()`, `Quit()` in listen mode, PDF page scale (§6.5)
`upstream/VEUSZ_UPSTREAM_ISSUE.md`	—	Ready-to-file issue text for the Veusz project, with the test results
`upstream/check_veusz_patch.py`	0.0.6	Reproduces the three Veusz limits and checks the patch, for any set of Veusz programs
`tools/build_readme_pdf.py`	0.0.6	Rebuilds the README PDF and `.tex` (pandoc + LuaLaTeX)
`examples/simulated_session_20261012_005651/`	0.0.6	Output of a 40 s **simulated** session (four commanded frequency and power steps, 10 Hz): `quicksyn_session.vszh5`, the bookmarked `quicksyn_session.pdf`, the overview PNG, `quicksyn_session_report.tex` and `.pdf` (LuaLaTeX), `reduction_summary.md`. No instrument was connected

0.0.5 was never released as an archive; everything listed for 0.0.5 in the history (§11) ships for the first time in this package.

```
mamba env create -f environment_quicksyn.yml
mamba activate quicksyn
python quicksyn_scpi_gui_v0_0_6.py            # Help > About shows author, company, email, licence
python quicksyn_scpi_gui_v0_0_6.py --version  # prints the same identity block, no display needed
python tests/run_all.py                       # no instrument or display required
```

To include the checks that need a real Veusz program, point `QUICKSYN_TEST_VEUSZ_EXE` at it first (for example `veusz.exe`). The LuaLaTeX checks run when `lualatex` is found (§6.7).

Tick **Simulator mode** on the Connection tab to exercise the whole application, logging included, with no hardware attached.

---

2. Link parameters

Fixed by the vendor and not configurable on USB [1].

Parameter	Value
Baud rate	115200
Data bits / parity / stop bits	8 / none / 1
Flow control	none
Terminator	CR, decimal 13, `0x0D` — every command, every interface
Device buffer	**64 bytes including the terminator**
SCPI interfaces	USB, Ethernet, GPIB, RS232 (SPI is native-only)
Minimum firmware for SCPI	100

`DIAG:BAUD` retargets the **debug port** used by RS232, GPIB and Ethernet. It does *not* change the USB virtual COM port, so leave the baud selector at 115200 for a USB connection.

---

3. SCPI coverage

Every command in [1, Table 5] is implemented. “Verified” means the formatted byte string was asserted against the vendor’s own worked example.

SCPI	Driver method	Range / argument	Verified
`FREQ` / `FREQ?`	`set_frequency` / `get_frequency`	integer mlHz	✔
`POW` / `POW?`	`set_power` / `get_power`	±XX.X dBm, 0.1 dB granularity	✔
`OUTP:STAT[?]`	`set_rf_output` / `get_rf_output`	`ON`/`OFF`	✔
`OUTP:BLAN[?]`	`set_blanking` / `get_blanking`	`ON`/`OFF`	✔
`OUTP:ROSC:STAT[?]`	`set_reference_output` / `get_reference_output`	`ON`/`OFF`	✔
`ROSC:SOUR[?]`	`set_reference_source` / `get_reference_source`	`INT`/`EXT`	✔
`FREQ:LRSTAT[?]`	`set_lock_recovery` / `get_lock_recovery`	`ON`/`OFF`	✔
`PULM:STAT[?]`	`set_pulse_modulation`	`ON`/`OFF`	✔
`AM:STAT[?]`, `AM:DEPT[?]`	`set_am`, `set_am_depth`, `get_am_depth`	DAC 0–4095	✔
`FM:STAT[?]`, `FM:MODE[?]`, `FM:SENS[?]`	`set_fm`, `set_fm_mode`, `set_fm_sensitivity`	mode 1–4, DAC 0–4095	✔
`DIAG:CAL:REF:DAC[?]`	`set_reference_dac` / `get_reference_dac`	DAC 0–65535	✔
`*RST`, `*SAV`, `*RCL`	`reset`, `save_state`, `recall_state`	slots 1–2 / 0–2	✔
`*IDN?`	`identify`	five comma-separated fields	✔

SCPI	Driver method	Range / argument	Verified
`STAT?`	`get_status`	8-bit field, lock bits inverted	✔
`DIAG:MOD?`	`get_modulation_status`	6-bit field	✔
`DIAG:MEAS? 21`	`get_temperature`	°C	✔
`LIST:PVEC`, `LIST:PVEC:RUN`	`write_list_point`, `run_list_point`	point 1–32767	✔
`LIST:SETUP`, `LIST:STAR`, `LIST:STOP`	`setup_list`, `start_list`, `stop_list`	runs 0–32767	✔
`LIST:SAV`, `LIST:ERAS`	`save_list`, `erase_list`	—	✔
`SWE:FAST:FREQ:SETUP` / `:STAR`	`setup_fast_frequency_sweep` / `start_sweep`	≤ 32767 points	✔
`SWE:NORM:FREQ:SETUP` / `:STAR`	`setup_normal_frequency_sweep` / `start_sweep`	explicit step	✔
`SWE:FAST:POW:SETUP` / `:STAR`	`setup_fast_power_sweep` / `start_sweep`	≤ 500 points	✔
`SWE:NORM:POW:SETUP` / `:STAR`	`setup_normal_power_sweep` / `start_sweep`	explicit step	✔
`SWE:STOP`	`stop_sweep`	—	✔
`DIAG:BAUD[?]`	(exposed, debug port only)	baud	—

Of the IEEE 488.2 common commands [6] the QuickSyn implements only `*IDN?`, `*RST`, `*SAV` and `*RCL`. There is **no** `*CLS`, `*ESR?`, `*OPC`, `*STB?` or error queue, so command acceptance can be confirmed only by reading the affected parameter back. The application does exactly that after every reset and on demand from **Instrument → Read all settings**.

3.1 Mandatory settling delays

Applied automatically by the driver, from [1, Tables 2a/2b].

After	Wait
`*RST`	2 ms
`*SAV`	100 ms
`*RCL`	50 ms
`LIST:ERAS`	200 ms
`LIST:PVEC` with `,F` (flash)	300 ms
`LIST:PVEC` RAM only	100 µs
`LIST:SAV`	50 ms + 2.5 ms per point
`DIAG:CAL:REF:DAC`	a few seconds
`FREQ` while FM is on	1 ms

3.2 Interlocks enforced

- Lists and sweeps are refused while FM is on [1, Tables 2b/2c].
- Dwell times are quantised to the mandatory **5 µs** increment.
- Power is quantised to 0.1 dB, the command granularity.
- Frequency and power are range-checked against the detected model.
- Every command is pre-flighted against the 64-byte buffer.
- The Modulation tab is hidden for models with no PULSE/AM/FM inputs.

4. The 64-byte buffer, and why frequencies are encoded two ways

Millihertz is the documented default unit for `FREQ` [1, Table 5], and a standalone `FREQ` is sent that way because it is exact and always fits. But a 20 GHz carrier is 14 characters in millihertz, and

```
SWE:FAST:FREQ:SETUP 2000000000000,10000000000000,80,2.0dBm,1s,10,0,0
```

is **69 bytes with its terminator** — five over the device buffer. This is why every compound example in [1] writes frequencies with a `GHz` suffix. `format_frequency()` therefore returns the shortest *exact* spelling among millihertz, `KHz`, `MHz` and `GHz`:

Requested	Encoded	Why
2.000000000 GHz	`2GHz`	4 bytes beats 13
2.200000000 GHz	`2.2GHz`	matches the `FREQ 2.2GHz` example in [1]
0.100000000 GHz	`0.1GHz`	6 bytes beats `100MHz`, ties go to GHz
9.111222333 GHz	`9111222333000`	13 bytes beats `9.111222333GHz` at 14

The same sweep now encodes to 51 bytes. A command that still will not fit — three full-precision frequencies in a normal sweep, say — is refused *before* transmission with advice on which parameter to round, rather than being silently truncated by the device.

---

5. Logging

Each session creates `<log_root>/<YYYYmmdd_HHMMSS>_<model>_<opts>/`:

Artefact	Contents
`session_metadata.json`	Identity, capability row, transport settings, sampler configuration, record dtype, versions
`session.log`	Human-readable event log
`transactions.csv`	`seq, iso_utc, t_unix, direction, command, response, latency_ms, ok, error` — every byte exchanged
`state_events.csv`	`seq, iso_utc, t_unix, parameter, value, source` — every commanded change of generated-signal state
`telemetry.csv`	25 columns: timestamps, frequency, power, temperature, decoded status and modulation bits, latency, instantaneous rate
`telemetry_chunk_####.pkl.gz`	gzip-pickled NumPy blocks of the same records, 52 bytes each (written via `.part` + rename)
`quicksyn_session.vszh5`	Veusz HDF5 document: all data and plot pages (§6) — written by the controller at stop, on demand, or every N minutes

`transactions.csv` is the audit trail of *what was sent*; `state_events.csv` is the readable record of *what was generated*; `telemetry.csv` is *what the instrument reported back*, at the sample rate.

5.1 Sample rate

The requested period is

$$\tau_0 = \frac{1}{f_s}$$

and each sample costs one serial round trip per enabled field. With N_q queries per sample and mean round-trip time t_q the ceiling is

$$f_{s,\max} = \frac{1}{N_q\, t_q}$$

Polling frequency, power, status, modulation and temperature gives $N_q = 6$ — temperature counts twice while the conservative double-issue rule is enabled — so at $t_q \approx 8$ ms,

$$f_{s,\max} \approx \frac{1}{6 \times 8 \times 10^{-3}} \approx 21 \text{ Hz}$$

The Monitor tab shows this budget live and turns it into a warning when the requested rate exceeds it; the **achieved** rate and the serial duty cycle are measured once per second, not estimated. Temperature decimation (every *N*th sample, default 5) buys the bandwidth back, since the internal temperature moves on a thermal time constant of minutes.

Scheduling is absolute: the deadline for sample k is computed from the acquisition start time as $t_0 + k\,\tau_0$, so jitter in one iteration does not accumulate into drift.

5.2 Memory discipline

Mechanism	Effect
Preallocated NumPy ring buffer	Fixed RAM: `capacity × 52` bytes, allocated once (60 000 records ≈ 3.1 MB)
Spill on overflow	A full block is handed to the writer thread and the index resets, so steady-state RAM is flat however long the run
Queue-decoupled writer thread	All disk I/O leaves the sampler thread; a slow filesystem cannot distort the sample interval
Flush RAM to disk now	Spills the resident block on demand
Live accounting	Ring fill, buffer RAM, process RSS, writer queue depth and chunk count on the Monitor tab

5.3 Parallelism and GPU

Device I/O cannot be parallelised — the instrument is one half-duplex endpoint behind a single reentrant lock. What *is* parallel: the sampler thread, the writer thread, and the GUI, plus process-pool chunk loading and optional CuPy reductions in the companion script. The controller uses no GPU on purpose: at a ~20 Hz ceiling the host-device transfer would cost more than the arithmetic it replaces.

5.4 Live SCPI console

View ▸ SCPI console (**Ctrl+Shift+C**) shows every exchange with the instrument as it happens, in a pane docked under the tabs with room for about ten lines. It can be resized with the splitter, undocked, moved to any edge or closed, and opens again from the View menu. To make that room, the tab pages now scroll when the window is shorter than a page: in 0.0.4 the tallest page (Monitor and log) forced a minimum window height of 981 px, which would not fit a small laptop screen. The window opens at 1180 × 960 px, or smaller if the screen is.

```
19:45:19.229 GUI  → POW -3.0  (write, 0.6 ms)
19:45:19.231 GUI  → FREQ?
                  ← 12500000000000  (1.6 ms)
19:45:21.652 GUI  → FREQ 9876543210000  (write, 0.6 ms)
19:45:21.653 GUI  → POW?
                  ✖ empty response (timeout)  (500.0 ms)
19:45:22.643 POLL → FREQ?
                  ← 9876543210000  (1.6 ms)
```

(Lines copied from the 0.0.6 console in a simulated session; the timeout was injected. The `POLL` line is visible because *Show telemetry polls* was ticked.) After a long pause with polls shown, the redraw starts with one grey line, for example after 200 200 exchanges:

```
⋯ 197700 earlier exchanges not shown here (the console keeps the newest 5000 lines); the session's
 ↪  transactions.csv holds every exchange while logging
```

Each line starts with the local time to the millisecond and the source. A command that expects no reply is one line with its write time; a query is the command and, under it, the reply or the error.

Element	Meaning	Colour
→	Command sent, exactly as transmitted (terminator not shown)	blue
←	Reply received, with the round-trip latency	green
✖	Timeout or I/O error, with the reason	red
`GUI` / `POLL`	Issued by you (any tab or menu) / by the telemetry sampler	grey / amber

The colours are mid-tones chosen to stay legible in the light and dark themes.

Control	Effect
Show telemetry polls	Off by default: at 10–20 Hz the sampler's `FREQ?`/`POW?`/`STAT?` traffic would bury your own commands. Polls are still counted and still logged to `transactions.csv`; ticking the box redraws the retained history with them
Filter	Shows only exchanges whose command, reply or error contains the text, e.g. `POW` or `LIST`. Applied when typing pauses for 250 ms, or at once on **Enter**. While paused it searches the frozen view
Pause / Resume	Freezes the view; exchanges keep arriving and the newest appear on **Resume**. A pause of any length costs no extra memory, and Resume draws at most the 5000 lines the view can hold, with a grey ⋯ line counting anything older
Auto-scroll	Follows the newest line while the view is at the bottom
Copy / Save… / Clear	Copy the view, save it as a `.txt` file, forget everything shown

The status line counts exchanges, polls and errors since the last clear.

How it is built. The driver reports every transaction through a Qt signal; exchanges made on the sampler thread cross to the GUI thread as queued signals (the slot runs when control returns to the GUI thread's event loop [24]), so no widget is touched from a worker thread. Receiving an exchange

only files it, in constant time. A 150 ms timer draws whatever is new, so a 20 Hz poll stream costs about seven redraws per second, not one per exchange. Lines go in through a `QTextCursor` with one character format per colour [25], measured at 11–21 µs per line against 142–186 µs for `appendHtml` (5000–10 000 lines); that is also why replies line up exactly under their commands, where HTML had collapsed the space after `GUI`.

Bounds, whatever the session length and however long the view is paused:

Bound	Value	Why
Lines in the view	5000 (oldest removed first by `maximumBlockCount` [23])	The text widget's own memory and layout cost
Exchanges kept for re-filtering	20 000 **per source** (GUI and POLL)	Polls can never push your own commands out of what the filter searches
Lines drawn per redraw	≤ 5000, the rest summarised in one ⋯ line	Resume after any pause, or a filter change, blocks the window for well under a second
Memory while paused	measured 5.3 MB for 200 200 exchanges (0.0.5: 45.5 MB, growing without limit)	Nothing is queued: Resume draws from the two stores

0.0.5 queued every exchange that arrived during a pause and drew all of them on Resume. At the default 2 Hz sample rate (3.4 queries per sample) 200 000 exchanges is about 8 hours of pause; at the nominal 50 Hz maximum, about 20 minutes. Measured on the same simulated window:

200 200 exchanges while paused	0.0.5	0.0.6
Python memory held (tracemalloc)	45.5 MB (unbounded)	5.3 MB (bounded)
Resume, polls hidden	0.16 s	0.01 s
Resume, polls shown	**62 s** with the window frozen	0.07 s
Typing three filter characters, full store	4.7 s of redraws	one redraw after the typing pause
Command / reply arrow columns	`GUI` 17 / 18 (one off), `POLL` 18 / 18	18 / 18 for both

The console is a live view, **not** the record — `transactions.csv` (§5) remains the complete audit trail of every byte exchanged, and the ⋯ line says so.

---

6. Veusz documents, reduction and plots

```
python quicksyn_veusz_export_v0_0_6.py <session_dir>
    --time-axis A      local (default), utc or elapsed
    --no-render        write the .vszh5 but skip the PDF/PNG render
    --png-pages P      first (default, the overview), all or none
    --veusz-exe PATH   Veusz program for rendering (default: search)
    --render-method M  auto (default), program or inprocess
    --no-compress      store .vszh5 datasets uncompressed
    --no-veusz         statistics only: no .vszh5, no plots
    --no-gpu           force the NumPy host path
    --workers N        process-pool size (default: CPUs this process may use)
    --dpi N            PNG resolution (default 150)
    --no-latex         do not write or compile the LuaLaTeX report
```

```
    --lualatex PATH    lualatex to use (default: search, see section 6.7)
    --version, -V      print version, copyright and licence, then exit
```

Writes `reduction_summary.json`, `reduction_summary.md` (Obsidian-ready, LaTeX equations), `telemetry_reduced.npz`, and **`quicksyn_session.vszh5`**, a native Veusz HDF5 document written directly with `h5py` — **Veusz is not needed to create it**. Open it in Veusz (File ▸ Open) to restyle the plots. When Veusz is installed, `quicksyn_session.pdf` (every page, with a bookmark hierarchy) and page PNGs are rendered *from* the `.vszh5`, which also proves it opens, and the LuaLaTeX report `quicksyn_session_report.tex` is written and, when `lualatex` is found, compiled to `quicksyn_session_report.pdf` (§6.7). No text `.vsz` is ever written. The controller produces the same files itself (§6.4).

6.1 Why `.vszh5` and not `.vsz`

A text `.vsz` stores embedded data with `fmt='%e'` [18], i.e. as $m \times 10^k$ with $1 \le m < 10$ printed to six decimals: seven significant digits. The exporter stores the carrier in GHz, so with $k = \lfloor \log_{10} f_{\mathrm{GHz}} \rfloor$ the quantisation step and the worst-case rounding error are

$$q = 10^{\,k-6}\ \mathrm{GHz}, \qquad |\varepsilon| \le \frac{q}{2}, \qquad \frac{|\varepsilon|}{f} \le \frac{q/2}{10^k} = 5 \times 10^{-7}.$$

Carrier	k	Step q	Worst-case error
1 to < 10 GHz	0	1 kHz	500 Hz
10 to < 100 GHz	1	10 kHz	5 kHz

Measured on 200 000 random carriers per decade: maximum 500.0 Hz and 5000.0 Hz, as predicted. Examples: 9.876543210 GHz came back 210 Hz off, 11.123456789 GHz 3211 Hz off, and two carriers 1 mHz apart merged. The `.vszh5` stores IEEE-754 float64 verbatim (53-bit significand, $\approx 1.1 \times 10^{-16}$ relative), so every value round-trips bit-exact.

6.2 The writer: `vszh5_writer.py` (from `isp_speed_monitor.py`)

The Veusz file creation is the method from the author's `isp_speed_monitor.py` 1.3.0 [21], moved into its own module so both the controller and the export script use one implementation, and so it can be reused in other programs. Your call signatures are kept.

Item	Origin	What it does
`VeuszWriter`	`VeuszWriter`	Opens `<file>.part`, writes `/Veusz` with `vsz_format = 1`, typed datasets (`dataset_1d` with `serr`, plus `perr`/`nerr`; `dataset_date`; `dataset_text`) and tags; `finish()` stores the script and renames atomically
`VszDoc`	`VszDoc`	Builds the Veusz command script: `add`, `to`, `up`, `set` (paths like `x/label`)
`vz_text()`	`vz_text()`	Escapes label text — now identical to Veusz `latexEscape`
`epoch_to_veusz()`, `local_utc_offsets()`	`epoch_to_veusz()`	Unix seconds → Veusz dates (s since 2009-01-01); per-sample UTC offset, DST-aware
`find_veusz_exe()`	`find_veusz_exe()`	`QUICKSYN_VEUSZ` / `VEUSZ_EXE`, PATH, then the usual install folders

Item	Origin	What it does
`render_pages()`	`tests/render_in_veusz.sh`	Drives the Veusz program in `--listen` mode with `loadHDF5Doc`; PDF of every page and per-page PNGs; falls back to the Veusz Python package when the program is absent or fails; bookmarks the PDF
`add_pdf_bookmarks()`	new in 0.0.6	Writes a bookmark hierarchy and document information into a PDF with pypdf, atomically, and opens the bookmark pane

Changes from 1.3.0, each found by testing against real Veusz:

Change	Reason
`vz_text` maps `\` to `{\backslash}` and escapes `[ ]`	In Veusz text `\\` is a **line break** [18, `textrender.py`], so the 1.3.0 escape split a label at any backslash (a Windows path in a title)
Atomic `.part` write, `abort()` and a context manager built into the class	An exception mid-write discards the partial file; the previous complete file stays
Optional gzip + shuffle	8-hour sessions; Veusz reads compressed datasets transparently
`render_pages` waits for a done marker, then ends Veusz	Veusz 4.2.1 does **not exit** on `Quit()` in listen mode (3.6.2 does); `render_in_veusz.sh` would sit until its `timeout 180`
Paths in the render script go through `repr()`	Windows back-slashes in an interpolated path would become escape sequences
0.0.5: the PNG path is bound on its own line, outside the `exec()` string	0.0.4 put it inside the `exec()` string, where it was unescaped a second time: `C:\Users\…` failed to compile, `C:\data\new` gained a newline, and a quote in the path broke the string. Found while fixing the same code in the speed monitor
0.0.5: `method="auto"` falls back to the Python package when the program *fails*, not only when it is missing	A Veusz on the PATH can be present but broken: Ubuntu's `/usr/bin/veusz` starts `/usr/bin/python3`, and when that has been switched to another Python than the one the package was built for (3.13 instead of 3.12 in the test machine) it stops with `No module named 'PyQt5.sip'`. Both errors are reported if the fallback fails too
0.0.5: a program that exits before reading its script, or cannot be started, is a failed render	It raised `BrokenPipeError` or `OSError` instead of `RuntimeError`, which skipped the fallback; a race, so it only showed up sometimes
0.0.6: PDFs are exported with `pdfdpi=72`	Veusz 4.2.1 drew every PDF page in the top-left 48 % of the page (§6.5). The PNGs were always right, and the 0.0.4 checks inspected PNGs, so it went unnoticed until the PDFs were bookmarked and included in the LaTeX report
0.0.6: the PDF gets a bookmark hierarchy	Session ▸ Overview, Time series (one entry per quantity), Stability (§6.7)
0.0.6: a re-render reports its files	0.0.5 compared against the folder listing taken before the render, so rendering a second time into the same folder returned nothing and the controller showed no plots

6.3 What the session document contains

Datasets	Tag	Contents
`raw/time_utc`, `raw/time_local`	raw	Veusz dates of every sample (local follows DST)
`raw/elapsed_s`, `raw/epoch_s`, `raw/utc_offset_s`, `raw/index`	raw	Time bases and sample index
`raw/freq_hz`, `raw/freq_ghz`, `raw/power_dbm`, `raw/temp_c`	raw	Reported carrier, level, internal temperature
`raw/latency_ms`, `raw/interval_ms`, `raw/rate_hz`	raw	I/O latency, sample interval, requested rate
`raw/status_word`, `raw/modulation_word`, `raw/rf_on`, `raw/rf_unlocked`, `raw/ref_unlocked`, `raw/voltage_error`	raw	`STAT?` / `DIAG:MOD?` words and decoded flags
`events/*` (times, `parameter`, `value`, `source`, `freq_ghz`, `power_dbm`)	events	Every commanded change from `state_events.csv`
`stability/tau_s`, `stability/adev`	stability	Overlapping ADEV of the reported carrier
`meta/session`, `meta/session_metadata_json`, `meta/reduction_summary_json`, `meta/generator`	meta	Provenance

Page	Contents
`page_overview`	3 × 2 grid: carrier, level, temperature, latency, sample interval, ADEV
`page_carrier`, `page_level`	Reported trace with **commanded values overlaid as markers**
`page_thermal`, `page_latency`, `page_interval`	One quantity per page against the chosen time axis
`page_stability`	Log–log ADEV

6.4 From the controller

The controller writes the `.vszh5` itself — no separate step:

Trigger	Where
Logging stops (default on)	Monitor tab ▸ Veusz document ▸ *Write when logging stops*
On demand, during or after logging	*Write .vszh5 now*, Veusz menu, **Ctrl+E**
Every N minutes while logging	Monitor tab ▸ *Refresh while logging* (0 = off)
PDF/PNG and the LuaLaTeX report as well	*Also render PDF/PNG with Veusz*, or Veusz ▸ *Write .vszh5 and render*; the status bar names the plots and the report
View	*Open in Veusz* (uses `find_veusz_exe`, else the desktop file association)

The export runs the companion script **as a separate process**: the GUI's PySide6 never shares a process with the Qt binding Veusz uses, a long session never stalls the window, and the export started by closing the window runs detached and finishes after the window is gone. While logging, the RAM

ring is spilled first and the writer thread allowed to commit it, so the document holds every sample up to that moment; telemetry chunks are now written to `.part` and renamed, so a concurrent export never reads half a chunk.

6.5 Rendering, and Veusz command-line limitations

Tested with Veusz 3.6.2 and 4.2.1:

- `veusz --export=out.pdf file.vszh5` **cannot** open a `.vszh5`: the command line loads every file as a text script, gets an empty document, and the export fails. The embedded interface's `Load` command has the same limit.
- `--export-dpi` is not a Veusz option (exit code 2), and `%PAGE%` page templates are expanded only by the GUI's Export dialog, not the command line.
- Veusz 4.2.1 does not exit on `Quit()` in `--listen --quiet` mode (3.6.2 does), so `render_pages()` waits for its own end marker, gives Veusz two seconds, then ends it.
- Veusz 4.2.1 draws a scripted PDF at 72/150 of its size, in the top-left corner of each page. `AsyncExport.getDPI()` rounds the PDF resolution to the only one Qt's PDF engine reports (72) and lays the page out at it, but `ExportPDFRunnable` sets the printer to the requested `pdfdpi`, and `Export()` defaults to 150 [18], [22]. Veusz 3.6.2 did not round, so it is correct. `render_pages()` passes `pdfdpi=72`, which is right for both.
- What works: `veusz --listen --quiet` fed a script that calls `veusz.document.loader.loadHDF5Doc` and `Export(..., page=[i])`, which is what `render_pages()` does, or the Python package's `Document.load(path, mode='hdf5')`.

Fixing it in Veusz. All three limits come from short functions in Veusz itself [18], unchanged on its `master` branch as of 2026-10-11 [22]:

1. `CommandInterpreter.Load()` opens every file as UTF-8 text, and both the command line and the embedding API go through it.
2. The listen loop's `quitProgram()` only closes a window that `--quiet` never showed. Qt ends the application on `lastWindowClosed`, which is emitted only for the last *visible* window [29].
3. `ExportPDFRunnable` sets the printer to `pdfdpi` rather than to the resolution the page was laid out at.

The patch `upstream/veusz-4.2.1-vszh5-load-quit-pdfdpi.patch` makes `Load()` recognise an HDF5 file by its signature and open it with `mode='hdf5'`, makes `quitProgram()` also quit the application, and sets the PDF printer to the layout resolution. A ready-to-file issue text is in `upstream/VEUSZ_UPSTREAM_ISSUE.md`.

`upstream/check_veusz_patch.py` reproduces all three on any set of Veusz programs. "Text reaches" is the rightmost text on page 1 as a fraction of the page width; about 0.97 means the drawing fills the page.

Check	4.2.1	4.2.1 + patch	3.6.2
`--export .vszh5` to PDF	fails (exit −11, segmentation fault, no file)	OK, text reaches 0.97	fails (exit 0, no file)
`--export .vszh5` to PNG	fails (exit −11, no file)	OK	fails (exit 0, no file)
`Load() .vszh5` + `Export()` of 2 pages	fails (no PDF)	OK, 2 pages, text reaches 0.97	fails (no PDF)
`Quit()` in `--listen --quiet`	still running 30 s after `DONE` (killed)	exits 0.12 s after `DONE`	exits 0.11 s after `DONE`
Text `.vsz --export` to PDF (regression)	OK, but text reaches **0.47**	OK, text reaches 0.97	OK, text reaches 0.97
This package's `check_vszh5.py`	58 / 58	58 / 58	58 / 58

The patch is for submitting upstream [22]; this package does **not** need it. `render_pages()` uses the listen-mode route and `pdfdpi=72`, which work with patched and unpatched Veusz alike.

6.6 Statistics

Reported statistics include achieved rate, interval jitter, I/O latency percentiles, distinct commanded frequencies, power and temperature extrema, margin to the +75 °C internal limit [2], lock-loss and voltage-error tallies, and the overlapping Allan deviation of the reported carrier per IEEE Std 1139-2008 [9]:

$$\sigma_y^2(\tau) = \frac{1}{2\,(N-2m+1)\,\tau^2} \sum_{k=1}^{N-2m+1} \left(x_{k+2m} - 2x_{k+m} + x_k\right)^2, \qquad \tau = m\,\tau_0$$

with phase residuals $x_k = \tau_0 \sum_{i<k} y_i$ and fractional error $y_i = (f_i - f_c)/f_c$. Evaluating it from cumulative sums turns the inner double sum from $\mathcal{O}(Nm)$ into $\mathcal{O}(N)$ per averaging factor; the result was cross-checked against a direct summation and agrees to better than 10^{-9} relative.

> **Read this statistic correctly.** `FREQ?` returns the *commanded* setting, not a measurement of the output. The ADEV here characterises command fidelity and the logging time base — a non-zero value means commands were lost, retimed or applied late. It is **not** carrier phase noise, which is specified separately in [4] and cannot be inferred from this log. Measuring the carrier needs a counter or a phase-noise analyser.

6.7 PDF bookmarks and the LuaLaTeX session report

Bookmarks in the rendered PDF. `render_pages()` writes the outline with `vszh5_writer.add_pdf_bookmarks()` (pypdf [26]) after Veusz has rendered the pages, sets the title, subject and keywords, and opens the bookmark pane on load. The export script supplies this hierarchy:

```
QuickSyn session <session folder>
├── Overview (all quantities)
├── Time series
│   ├── Carrier frequency
│   ├── Output level
│   ├── Internal temperature
│   ├── Serial I/O latency
│   └── Sample interval
└── Stability
    └── Allan deviation of the reported carrier
```

Without pypdf the PDF is still written, without an outline, and the export says so.

The LuaLaTeX report (`quicksyn_session_report.tex`, compiled to `quicksyn_session_report.pdf`) is the LaTeX version of the session PDF. It contains:

- the instrument identity;
- the timing, signal and thermal tables;
- the stability statistics with their equations;
- every Veusz page, included as vector graphics straight from `quicksyn_session.pdf`;
- the session's files and IEEE-style references.

hyperref [27] turns the sections and subsections into the bookmark hierarchy (Contents; 1 Instrument … 6 Plots ▸ 6.1 Overview … 6.7 Allan deviation; 7 Files; 8 Provenance; References) and opens the bookmark pane. A simulated session is flagged in a red box on the first page.

Item	Behaviour
When	Whenever the plots are rendered (not with `--no-render`, `--no-latex`, or the controller's periodic refresh)
Engine	LuaLaTeX [28], run twice for the contents and bookmarks; `.aux`/`.out`/`.toc` removed, `.log` kept

Item	Behaviour
Finding `lualatex`	`--lualatex PATH`, then `QUICKSYN_LUALATEX`, then the `PATH`, then `C:\texlive\<year>\bin\windows` (TeX Live 2023 and later; `bin\win64` before), then MiKTeX's folder
Packages used	`fontspec`, `amsmath`, `booktabs`, `array`, `graphicx`, `xcolor`, `fancyhdr`, `hyperref`, `bookmark`: all in a standard TeX Live or MiKTeX installation
No TeX installed	The `.tex` is still written; compile it later with `lualatex quicksyn_session_report.tex` (twice)
Render failed	The report is still written, with a note in place of the plots
Failure	The first LaTeX error line is printed and the `.log` kept; the data files are never affected

The equations in the report are the ones in §6.6 and §5.1, set as numbered LaTeX equations.

---

7. Model capability table

Transcribed from [4]. Frequency and power entry ranges clamp to the detected model on connect.

Model	Coverage	Opt. 1 low end	Resolution	Power	Opt. 2 control range	Mod. inputs	SCPI documented
FSW-0010	0.5–10 GHz	0.1 GHz	1 mHz	+15 dBm	−25 … +15 dBm (0.01 dB nom.)	yes	yes
FSW-0020	0.5–20 GHz	0.2 GHz	1 mHz	+13 dBm	−10 … +13 dBm (0.1 dB nom.)	yes	yes
FSL-0010	0.65–10 GHz	—	1 mHz	+15 dBm min	—	no	no
FSL-0020	0.65–20 GHz	—	1 mHz	+10 dBm min	—	no	no
FSL-E020	0.65–20 GHz	—	1 mHz	+10 dBm min	—	no	no
FSL-2740	27–40 GHz	—	1 Hz	+18 dBm typ	+17 … +24 dBm	no	no
FSL-5067	50–67 GHz	—	1 Hz	+17 dBm typ	+15 … +22 dBm	no	no
FSL-7682	76–82 GHz	—	1 Hz	+11 dBm typ	+10 … +14 dBm	no	no

[1] names only the FSW-0010 and FSW-0020. The other models are offered with the same commands but flagged **unverified** in the Connection tab, and an unrecognised model falls back to the union of all documented limits with a warning. Lock-recovery, blanking, reference routing and list/sweep commands are expected to be common across the family, but that is an inference.

---

8. Memory hygiene

Per [5]: the 2112-byte and 16 MB SRAM areas clear on a power cycle; the 8 MB flash has **no direct erase**, so the two user-writable areas can only be cleared by overwriting them with the same commands that wrote them.

Area	Procedure	Cost
Instrument states (2 slots)	`*RCL 0`, then `*SAV 1` and `*SAV 2`	~0.3 s
List point table (32767 locations)	Write safe frequency/power with `,F` into each location	300 ms each → **~2.7 h for all**

The Memory tab implements both, with an ETA and an abort. The cheaper course is prevention, which is why **RAM-only list download is this application's default**: it is ~3000× faster per point and the points clear on the next power cycle.

---

9. Documented conflicts and how they are handled

#	Conflict	Handling
1	Post-reset blanking: `ON` in [1, Table 2a] vs `off` in [2, Table 1]	Never assumed; `STAT?` is read back after every reset and the observed value reported
2	SCPI `*IDN?` field order undefined; the [1] example puts `0` in the version position, below the firmware-100 gate it requires	Labelled per the native order in [1, Table 4], **all five raw fields displayed verbatim**, gate is advisory and never blocks
3	Query double-issue required on SPI, temperature-only on native, unstated for SCPI	`TEMPERATURE_DOUBLE_QUERY`, default `True`, recorded in the session metadata
4	Response terminator unspecified	Reader accepts CR, LF or CRLF, times out cleanly
5	0.01 dB resolution quoted in [4] vs tenth-dBm command units in [1]	Quantised to 0.1 dB; the quantised value is logged
6	mlHz default vs GHz-suffixed examples in compound commands	Shortest exact spelling, pre-flighted against the buffer (§4)
7	`SWE:FAST:FREQ:SETUP` example in [1] has a missing comma	Read as the eight documented fields plus the optional `RUN`
8	"Start NF Sweep" example printed as `SWE:NORM:FREQ:SETUP 1`	Treated as a typo for `SWE:NORM:FREQ:STAR 1`
9	32767 addressable list points vs "32,000 points" in [4]	Command-set limit used for range checks, datasheet figure surfaced in the UI
10	`DIAG:BAUD` assumed to apply to USB	Documented as debug-port only; USB fixed at 115200

---

10. Verification status

Validated headlessly against the built-in simulator, with Qt on the offscreen platform. The first table was run for **this build** (0.0.6, 2026-10-11) on Python 3.12.3 (the declared target, clean virtual environment)

and Python 3.13.16, PySide6 6.12.0. The environment had:

- real Veusz **3.6.2** (Ubuntu 24.04 package, PyQt5 / Qt 5.15.13);
- real Veusz **4.2.1** (the version conda-forge ships), built here from the PyPI source against PyQt6 6.6.1 / Qt 6.4.2;
- the same 4.2.1 with the upstream patch of §6.5;
- **LuaLaTeX** from TeX Live 2023 (LuaHBTeX 1.17.0, hyperref 7.01h).

The 3.12 run is the one that counts: it found a defect the 3.13 runs had hidden (0.0.3 history entry). The second table lists results 0.0.6 did not change. Rows marked † are measurements from 0.0.4/0.0.5 that were not repeated. The others are re-run by the test suite in every build; their counts, from an earlier run, vary by a few samples with timing.

Check (0.0.6 build)	Result
`tests/run_all.py`, Python 3.12.3, `QUICKSYN_TEST_VEUSZ_EXE` = Veusz 4.2.1	5 / 5 checks pass (release metadata, smoke, reduction, `.vszh5`, console)
`tests/run_all.py`, Python 3.13.16	5 / 5 checks pass; `check_vszh5.py` skips the 5 in-process Veusz checks (no Veusz package for 3.13 here) and runs the program checks through Veusz 4.2.1
`check_release_metadata.py`	148 / 148: every file at 0.0.6 with a 0.0.6 history entry, SPDX tags in every Python file, `LICENSE` SHA-256, `--version`, About dialog and Help ▸ About, README PDF bookmarked and current
`check_vszh5.py` with real Veusz	**58 / 58** each with 4.2.1 (in-process and program), patched 4.2.1, and 3.6.2 (in-process and program)
PDF page scale (0.0.6 fix)	rightmost text at 0.97 of the page width on every page (0.89 on the log–log ADEV page) with 4.2.1, patched 4.2.1 and 3.6.2, program and in-process; the 0.0.5 writer with 4.2.1 gave 0.46–0.48, i.e. the drawing in the top-left corner
PDF bookmarks	10 bookmarks in 3 levels exactly as in §6.7; title and subject set; page mode `/UseOutlines`; a bookmark pointing outside the PDF is refused
Re-render into the same folder	returns the same 8 files (0.0.5 returned none)
LuaLaTeX report	compiles in 2.2 s (two runs) with **no LaTeX warnings**; 9–10 pages; bookmarks: Contents, 8 numbered sections, 7 plot subsections under 6 Plots, References; outline pane opens; `.aux`/`.out`/`.toc` removed; no unescaped underscore in the text; simulated session flagged; pages inspected as images
Export CLI with render	writes `quicksyn_session.pdf`, `quicksyn_session_report.tex` and `.pdf`; prints the report line the controller shows in its status bar
Example session (`examples/`)	40 s simulated session driven through the GUI at 10 Hz (401 samples, four commanded steps); export with Veusz 4.2.1: 7-page bookmarked PDF, 9-page LuaLaTeX report with 17 bookmarks (Contents, 8 sections, 7 plot subsections, References); pages inspected as images
README PDF	built with `tools/build_readme_pdf.py` (pandoc 3.1.3, LuaLaTeX): 29 pages, no missing glyphs, bookmarks for all 14 sections and their subsections
`.vszh5` written without Veusz	in an interpreter where `veusz` cannot be imported, `--no-render` writes the full document; `python -X importtime` lists no Veusz module

Check (0.0.6 build)	Result
`check_console.py`	all pass on 3.12.3 and 3.13.16: write/query/error/empty-reply/CR-LF lines; reply arrow under the command arrow (column 18) for GUI and POLL; 5000 exchanges from a worker thread all filed on the GUI thread; a simulated session's 142 exchanges = 142 rows of `transactions.csv`; 200 200 exchanges while paused hold 5.3 MB, Resume 12–106 ms, view ≤ 5000 lines with the … notice, all 200 own commands kept; filter debounced, Enter immediate, paused filter searches the frozen view
SCPI console, 0.0.5 vs 0.0.6 (same probe)	0.0.5: 45.5 MB held, 62 s frozen on Resume, 1.6 s per filter keystroke, GUI reply column one off; 0.0.6: figures in §5.4
`check_smoke.py`	commands, interlocks, sampler, ring spills, GUI session logging, the controller's `.vszh5` export mid-session and at stop (= `telemetry.csv`), capability gating
Upstream Veusz patch	`patch -p1 --dry-run` clean on the 4.2.1 source; results of `upstream/check_veusz_patch.py` in §6.5
Recovered `vszh5_writer.py` 0.0.5	rebuilt byte-for-byte from 0.0.4 plus the 0.0.5 changes kept in the project notes: SHA-256 `2cdf43d2…ae33c1c`, as recorded for 0.0.5
Archive	see §11; checked with 7-Zip 16.02 `t`, extracted to a clean folder and `run_all.py` re-run there

Check (unchanged in 0.0.6)	Result
`STAT? → 00A8` and `DIAG:MOD? → 10` decoding	matches [1, Table 5] bit maps, lock-sense inversion included
Unit conversions, power and dwell quantisation	pass
34 formatted commands vs the vendor worked examples	pass; longest 58 bytes of 64
`LIST:PVEC` against both [1, Table 2b] examples	byte-exact
Over-long compound command	refused with rounding advice
FM interlock on lists and sweeps	both blocked
8 out-of-range parameters	all rejected
Sampler at 20 Hz	40 samples in 2.0 s, 0 errors
Ring overflow, 250 records into capacity 100	spills of 100/100/50, flat RAM
GUI construction, 7 tabs, 3 themes	pass
End-to-end GUI session	3 commanded frequencies + power + RF state logged to `state_events.csv`, 11 telemetry rows, 62 transactions
Capability gating (FSL-2740)	Modulation tab hidden, entry clamped to 40 GHz
ADEV vs direct $O(Nm)$ summation	agrees to $< 10^{-9}$ relative over 4 averaging factors
Moving statistics vs `np.convolve`	agrees to 10^{-9}
Dataset types in real Veusz	`Dataset` (1d, with `serr`/`perr`/`nerr`), `DatasetDateTime`, `DatasetText`; tags restored
Data precision †	288 000-sample session bit-exact after loading in Veusz; carriers 1 mHz apart stay distinct; commanded values equal reported ones
Escaping	names with `/` and `` ` `` restored; `vz_text` identical to Veusz `latexEscape`; `Run_01 ^ {x} [y] C:\data` renders literally

Check (unchanged in 0.0.6)	Result
Local time across DST	offsets −4 h → −5 h across the 2026-11-01 change (America/New_York)
Pages	overview + 6 single pages, every trace pointing at an existing dataset, for local, UTC and elapsed axes
8-hour session, 10 Hz †	288 000 samples reduced, written (6.2 MB `.vszh5`) and rendered to PDF + 7 PNGs in ≈ 27 s
Controller export while logging	`.vszh5` rewritten mid-session (11 samples) without stopping acquisition; at stop it holds every sample (= `telemetry.csv`)
No text `.vsz`, no leftover `.part`	checked in the command-line run and the GUI session
Telemetry writer shutdown on Python 3.12	thread exits cleanly after `stop()` (regression for the `_stop` shadowing defect)
3.13-only API	no use of `os.process_cpu_count` (added in 3.13 [16]); `available_cpus()` uses scheduler affinity with an `os.cpu_count()` fallback

Not yet verified: the LuaLaTeX report with TeX Live 2025 or MiKTeX on Windows (verified with TeX Live 2023 on Linux; the packages it uses are standard in both); anything requiring hardware — real round-trip latencies, the actual response terminator, whether the SCPI temperature query needs its second issue and the `*IDN?` field order. The Veusz render is verified on Linux with Veusz 3.6.2 and 4.2.1, **not yet on Windows** in your conda-forge environment; in particular, whether the Windows `veusz.exe` (a windowed program) reads a script piped to its standard input in `--listen` mode is untested. If it does not, `method="auto"` falls back to the Python package, which conda-forge installs. Also not run: an actual `mamba env create` (the container has no conda-forge access). The presence of `qt.py` 2.0.3 and `veusz` 4.2.1, `h5py` 3.16.0 (win-64 included) and `pypdf` 6.20.0 (noarch) on conda-forge were confirmed from their package pages [17], [19], [26], which is weaker than a successful solve.

---

11. Revision history

Revision	Date	Change
0.0.1	2026-10-09	First internal draft. Transport, driver, capability table, simulator, ring buffer, sampler, writer thread, seven-tab GUI, theming, session logging.

Revision	Date	Change
0.0.2	2026-10-09	Line-by-line verification against [1]–[5]. Compound-command frequency encoding rewritten after the smoke test found every sweep setup overflowing the 64-byte buffer; dwell quantised to 5 µs; `*IDN?` relabelled with all raw fields shown and the firmware gate made advisory; `DIAG:BAUD` corrected to debug-port only; Modulation tab gated on model capability; ring-buffer floor documented; references corrected and extended; conflict list grown from 6 to 10 entries with exact citations; companion reduction and Veusz export script added.
0.0.3	2026-10-09	Release metadata. Author, company, email and the GPL-3.0-or-later notice added to every header, with SPDX tags, module dunders, a `--version` flag and an About dialog (version, author, company, `mailto:` link, licence, GPL notice, runtime versions); `LICENSE` and `tests/` added; template placeholders removed. **Fixed (Python 3.12 only):** `TelemetryWriter` kept its stop flag in `self._stop`, shadowing `threading.Thread._stop()`, so `join()` raised `TypeError: 'Event' object is not callable` when logging stopped — the 0.0.2 controller has this defect on 3.12. **Fixed (Python 3.12 only):** the export script defaulted its worker count with `os.process_cpu_count()`, which exists only from 3.13 [16]; it now uses `available_cpus()`. Corrected an inaccurate “PEP 695” remark in the interpreter-guard comment.

Revision	Date	Change
0.0.4	2026-10-11	**Veusz output is a `.vszh5` written with the author's method from `isp_speed_monitor.py` 1.3.0**, now the standalone module `vszh5_writer.py` (`VeuszWriter`, `VszDoc`, `vz_text`, `epoch_to_veusz`, `find_veusz_exe`, `render_pages`); no text `.vsz` (which kept 7 significant digits: up to 500 Hz error below 10 GHz, 5 kHz above). `export_veusz()` follows the speed monitor's layout: tagged `raw/`, `events/`, `stability/`, `meta/` datasets, UTC and local date axes, overview + per-quantity pages, commanded values overlaid. **The controller writes the `.vszh5` itself** at stop, on demand (Ctrl+E) and every N minutes while logging, via the export script in a separate process; Open in Veusz; atomic chunk files. Rendering drives the Veusz program in `--listen` mode (the command line cannot open `.vszh5`). Fixes found on the way: titles with _ printed as subscripts (0.0.3); commanded/reported GHz differed in the last bit; Veusz 4.2.1 listen mode does not exit on `Quit()`. Checked against real Veusz 3.6.2 and 4.2.1.

Revision	Date	Change
0.0.5	2026-10-11	*Not released as an archive; these changes ship in 0.0.6.* **Live SCPI console** (View ▸ SCPI console, Ctrl+Shift+C): every command and reply as it happens, with time, source (GUI or telemetry poll), latency and errors; colour-coded for light and dark themes; polls hidden by default; filter, pause, copy, save; thread-safe through a queued signal, batched rendering, bounded memory (§5.4). Tab pages scroll, so the window no longer has to be taller than the tallest page, and it opens at 1180 × 960 px clamped to the screen. **Render fixes** in `vszh5_writer.py`: the PNG path is bound outside the `exec()` string (a Windows path was unescaped twice); `method="auto"` falls back to the Python package when the Veusz program fails, not only when it is missing; a program that exits before reading its script or cannot start is reported, not raised as `BrokenPipeError`/`OSError`; the output pipe is closed and Veusz never left running. New tests for all four. **Veusz upstream patch** and issue text for the two Veusz limits (§6.5). **Documentation:** the licence question on the `isp_speed_monitor.py` code is closed (§13). History entries put in ascending order in every file.

Revision	Date	Change
0.0.6	2026-10-11	**First package since 0.0.4.** 0.0.5 was never released as an archive, so its changes ship here as well. **Live SCPI console bounded while paused:** 0.0.5 queued every exchange during a pause and drew them all on Resume. 200 000 exchanges (≈ 8 h at 2 Hz) held 45.5 MB and, with polls shown, froze the window for 62 s, during which even *RF output off (panic)* had to wait. Now nothing is queued: GUI and POLL keep separate stores of 20 000, so polls cannot push your own commands out of the searchable history. A redraw draws at most the 5000 lines the view holds, with a ⋯ notice for the rest. Lines are inserted with a text cursor (≈ 10× faster than `appendHtml`) and line up exactly. The filter waits for a 250 ms typing pause (Enter applies at once) and, while paused, searches the frozen view. CR/LF in a reply shows as ↵; an empty reply is labelled (§5.4). **PDF bookmark hierarchy** in the rendered session PDF (`add_pdf_bookmarks()`, pypdf). **LuaLaTeX session report**, the LaTeX version of the session PDF: statistics, equations, every plot page as vector graphics, references, with hyperref bookmarks (§6.7). New options `--no-latex` and `--lualatex`. **Fixed (Veusz 4.x):** PDF pages were drawn at 48 % scale in the top-left corner, because Veusz 4.2.1 lays a PDF out at 72 dpi but sets the printer to `Export()`'s default `pdfdpi=150`; PDFs are now exported with `pdfdpi=72`. The PNGs were always right, which is why the 0.0.4 checks, which inspected PNGs, missed it. **Fixed:** a second render into the same folder returned no files, so the controller showed no plots. **Veusz upstream patch** now fixes three limits (`.vszh5` in `--export`/`Load()`, `Quit()` in listen mode, the PDF scale), with a reproduction script (§6.5). **Tests:** all re-targeted to 0.0.6; new `check_console.py`; `check_vszh5.py` grows from 36 to 58 checks (render plumbing,

12. References

#	Reference	URL
[1]	National Instruments, “QuickSyn Microwave Synthesizers: Communications Specifications,” Doc. No. 5580510-01, Santa Clara, CA, USA, 2014. (Revision not printed.)	Supplied with this task
[2]	National Instruments, “QuickSyn Microwave Frequency Synthesizer: User Guide,” Doc. No. 5580513-01, Rev. F, ECN 001779.	Supplied with this task
[3]	National Instruments, “QuickSyn Microwave Frequency Synthesizer: Calibration Software User’s Manual,” Doc. No. 5580515-01, Rev. B, ECN 001540, 2014.	Supplied with this task
[4]	National Instruments, “QuickSyn Frequency Synthesizers,” product brochure and datasheet, Oct. 2017.	Supplied with this task
[5]	National Instruments, “Clearing QuickSyn Memory,” application note.	Supplied with this task
[6]	IEEE Std 488.2-1992, *IEEE Standard Codes, Formats, Protocols, and Common Commands for Use With IEEE Std 488.1-1987, IEEE Standard Digital Interface for Programmable Instrumentation*, IEEE, New York, NY, USA, 1992.	https://ieeexplore.ieee.org/document/8705775
[7]	IEC 60488-2:2004 / IEEE 488.2, *Standard digital interface for programmable instrumentation — Part 2: Codes, formats, protocols and common commands*, IEC, Geneva, Switzerland, 2004.	https://www.vde-verlag.de/iec-normen/preview-pdf/info_iec60488-2%7Bed1.0%7Den.pdf
[8]	SCPI Consortium, *Standard Commands for Programmable Instruments (SCPI), Volume 1: Syntax and Style*, Version 1999.0, May 1999 (now maintained by the IVI Foundation).	https://www.ivifoundation.org/downloads/SCPI/scpi-99.pdf

#	Reference	URL
[9]	IEEE Std 1139-2008, *IEEE Standard Definitions of Physical Quantities for Fundamental Frequency and Time Metrology — Random Instabilities*, IEEE, New York, NY, USA, 2008.	https://ieeexplore.ieee.org/document/4797525
[10]	IVI Foundation, “Specification Downloads” (SCPI and IVI specifications index).	https://www.ivifoundation.org/specifications/default.html
[11]	Veusz Developers, “Veusz — a scientific plotting package.”	https://veusz.github.io/
[12]	Phase Matrix, Inc., “Phase Matrix Introduces a New 20 GHz QuickSyn Frequency Synthesizer,” press release, IEEE MTT-S IMS 2010, Anaheim, CA, USA, May 2010.	https://www.rfcafe.com/miscellany/press-releases/2010/phase-matrix-5-20-2010.htm
[13]	Free Software Foundation, “GNU General Public License, version 3,” 29 Jun. 2007.	https://www.gnu.org/licenses/gpl-3.0.html
[14]	SPDX Workgroup, “GNU General Public License v3.0 or later (GPL-3.0-or-later),” SPDX License List.	https://spdx.org/licenses/GPL-3.0-or-later.html
[15]	T. Preston-Werner, “Semantic Versioning 2.0.0.”	https://semver.org/spec/v2.0.0.html
[16]	Python Software Foundation, “What’s New In Python 3.13” (adds `os.process_cpu_count()`).	https://docs.python.org/3/whatsnew/3.13.html
[17]	Anaconda, Inc., conda-forge package pages for `qt.py` and `veusz`.	https://anaconda.org/conda-forge/qt.py ; https://anaconda.org/conda-forge/veusz
[18]	Veusz Developers, “Veusz 4.2.1” source distribution: `veusz/document/doc.py` (`saveToHDF5File`), `veusz/document/loader.py` (`loadHDF5Doc`), `veusz/datasets/oned.py`, `veusz/utils/textrender.py` (`latexEscape`), `veusz/veusz_main.py`, Python Package Index.	https://pypi.org/project/veusz/4.2.1/
[19]	Anaconda, Inc., conda-forge package page for `h5py`.	https://anaconda.org/conda-forge/h5py
[20]	A. Collette *et al.*, “h5py 3.16.0,” Python Package Index.	https://pypi.org/project/h5py/
[21]	W. W. Wallace, *isp_speed_monitor.py*, version 1.3.0 (`VeuszWriter`, `VszDoc`, `vz_text`, `export_veusz`, `tests/render_in_veusz.sh`), Adaptive Electrodynamics LLC dba AdaptEd, Oct. 2026.	Supplied by the author (not public)

#	Reference	URL
[22]	Veusz Developers, “veusz/veusz,” GitHub repository: `master` branch (`veusz/document/commandinterpreter.py`, `veusz/veusz_listen.py`) and issue tracker, accessed 11 Oct. 2026.	https://github.com/veusz/veusz ; https://github.com/veusz/veusz/issues
[23]	The Qt Company, “QPlainTextEdit Class,” *Qt Widgets*, Qt 6.12 documentation (`maximumBlockCount`: blocks are removed from the beginning of the document), accessed 11 Oct. 2026.	https://doc.qt.io/qt-6/qplaintextedit.html
[24]	The Qt Company, “Threads and QObjects,” Qt 6.12 documentation, section “Signals and Slots Across Threads” (auto and queued connections), accessed 11 Oct. 2026.	https://doc.qt.io/qt-6/threads-qobject.html
[25]	The Qt Company, “QTextCursor Class,” *Qt GUI*, Qt 6.12 documentation (`insertText` with a `QTextCharFormat`, `insertBlock`, `beginEditBlock`), accessed 11 Oct. 2026.	https://doc.qt.io/qt-6/qtextcursor.html
[26]	pypdf contributors, “pypdf 6.20.0,” Python Package Index, 9 Oct. 2026 (BSD-3-Clause; also on conda-forge, noarch).	https://pypi.org/project/pypdf/ ; https://anaconda.org/conda-forge/pypdf
[27]	The LaTeX Project Team, H. Oberdiek and S. Rahtz, “hyperref: Extensive support for hypertext in LaTeX,” CTAN (LPPL 1.3; 7.01h used here, 7.01s current on 11 Oct. 2026).	https://ctan.org/pkg/hyperref
[28]	TeX Users Group, “The TeX Live Guide” (LuaLaTeX engine; Windows binaries in `bin/windows`, 64-bit since TeX Live 2023), accessed 11 Oct. 2026.	https://tug.org/texlive/doc/texlive-en/texlive-en.html ; https://tug.org/texlive/windows.html
[29]	The Qt Company, “QGuiApplication Class,” Qt 6 documentation (`lastWindowClosed`: emitted when the last *visible* primary window is closed), accessed 11 Oct. 2026.	https://doc.qt.io/qt-6/qguiapplication.html

Sourcing confidence

Claim class	Confidence	Basis
Link parameters, terminator, 64-byte buffer	**High**	Stated verbatim in [1]

Claim class	Confidence	Basis
SCPI mnemonics, arguments, ranges, bit maps	**High**	[1, Table 5], reproduced byte-exact in tests
Settling delays, 5 µs dwell increment, FM interlocks	**High**	[1, Tables 2a/2b/2c]
Model coverage, power, resolution, connectors, I/O levels	**High**	[4] specification tables
Memory layout and clearing procedure	**High**	[5], explicit
+75 °C internal limit	**High**	[2], CAUTION notice
`*IDN?` field semantics	**Low**	Inferred from [1, Table 4]; the [1, Table 5] example is self-inconsistent
SCPI temperature double-issue	**Low**	Documented for native and SPI only; conservative default chosen
Response terminator	**Low**	Not documented; reader tolerates CR/LF/CRLF
Non-FSW models accepting the same SCPI set	**Medium**	[1] covers FSW-0010/0020 only; flagged in the UI
GPL-3.0 text in `LICENSE`	**High**	Byte-identical to the Debian-packaged copy of the FSF text (674 lines); gnu.org itself could not be fetched, so [13] was not compared directly
`GPL-3.0-or-later` is a current, non-deprecated SPDX identifier	**High**	SPDX machine-readable record for [14]: `isDeprecatedLicenseId: false`, OSI-approved, FSF-libre
`os.process_cpu_count` is 3.13-only	**High**	[16], and observed directly: absent on 3.12.3, present on 3.13.16
`.vszh5` layout and loader behaviour	**High**	Read from the Veusz 4.2.1 source [18] and confirmed by loading, re-saving and rendering in real Veusz 3.6.2 and 4.2.1
Veusz `Load()` and listen-mode `Quit()` limits, and the patch fixing them	**High**	Reproduced on 4.2.1; patch tested before and after; the same two functions read on `master` [22] on 2026-10-11
Windows `veusz.exe` reading a piped script in `--listen` mode	**Low**	Not tested; the in-process fallback covers it
Veusz 4.x PDF page-scale defect and its cause	**High**	Reproduced on 4.2.1 (program, in-process, Xvfb and offscreen); cause read in the 4.2.1 source and on `master` [22]; fixed by the patch and by `pdfdpi=72`, both measured
PDF bookmarks and the LuaLaTeX report	**High** on Linux, **Medium** on Windows	Built and read back here (pypdf 6.20.0, TeX Live 2023); TeX Live 2025 / MiKTeX on Windows not run; the TeX Live folder layout is from [28]
Third-party licence strings (§14)	**High**	Read from the installed packages' own metadata, or from the PyPI / conda-forge pages

Claim class	Confidence	Basis
Third-party licence *compatibility* (§14)	**Medium**	My application of the FSF's published positions; gnu.org was not fetchable, and this is not legal advice
SCPI console bounds and timings (§5.4)	**High** for this build	Measured with the same probe on 0.0.5 and 0.0.6 (PySide6 6.12, offscreen, simulated transport); absolute times depend on the PC, the ratios should not
~8 ms round-trip estimate in the rate budget	**Medium**	Engineering estimate for a 115.2 kbaud USB-CDC bridge, not vendor data; the achieved rate is measured, not estimated

---

13. Open items

1. **Header layout.** Author, company, email and licence are as you specified; there is deliberately no project field. The field layout and the GPL notice block are my own design. If NRAO/GBO has a mandatory header template, send it and the next revision will apply it verbatim.
2. **Copyright holder and release approval.** The files name William W. Wallace as copyright holder, as instructed. Whether NRAO/GBO or its operator holds or must approve the copyright before public release is an institutional question this package cannot settle; check it before publishing.
3. **Hardware bring-up.** On first connection to a real unit, capture `transactions.csv` from a short session and the `*IDN?` line. That settles the field order, the response terminator, the real round-trip time, and whether the SCPI temperature query needs its second issue.
4. **Environment solve and Windows render.** Create the mamba environment on the Windows machine, log a short session and use Veusz ▸ *Write .vszh5 and render*; the render is verified on Linux only. If Veusz is a standalone install, point `QUICKSYN_VEUSZ` at `veusz.exe`. To see whether the program itself works there (rather than the in-process fallback), run the export script with `--render-method program`.
5. **Veusz upstream.** Filing `upstream/VEUSZ_UPSTREAM_ISSUE.md` with the patch at [22] needs a GitHub account. Once a Veusz release carries the fixes, plain `veusz --export=out.pdf file.vszh5` will work and draw full size. The listen-mode route and `pdfdpi=72` can stay for older versions.
6. **LuaLaTeX on Windows.** Render one session with TeX Live 2025 installed and check that `quicksyn_session_report.pdf` appears beside the plots. If `lualatex` is not on the PATH, set `QUICKSYN_LUALATEX`.

Closed in 0.0.6. *The full package:* 0.0.6 contains every file at one version; the files that existed only in the unreleased 0.0.5 build (tests, environment file, upstream patch) were rebuilt and re-verified (§10).

Closed in 0.0.5. *Licence of the code taken from `isp_speed_monitor.py`.* You own Adaptive Electrodynamics LLC dba AdaptEd, the copyright holder of `isp_speed_monitor.py`, so including the writer here under GPL-3.0-or-later is your own decision and needs no one else's consent. Both copyright lines stay in the `vszh5_writer.py` header. Licensing the same code under the GPL here does not change the terms of the speed monitor itself: a copyright holder may license its own code differently in different products. (Not legal advice.)

---

14. Licence

Copyright (C) 2026 William W. Wallace wwallace@nrao.edu, NRAO/GBO.

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version. It is distributed in the hope that it will be useful, but **without any warranty**; without even the implied warranty of merchantability or fitness for a particular purpose. See the full text in `LICENSE` [13].

SPDX identifier: `GPL-3.0-or-later` [14]. Each source file starts with `SPDX-License-Identifier` and `SPDX-FileCopyrightText` tags and repeats the FSF-recommended notice in its header. The notice also appears in the About dialog (Help > About) and the identity lines in `--version`.

`vszh5_writer.py` additionally carries `SPDX-FileCopyrightText: 2026 William W. Wallace, Adaptive Electrodynamics LLC dba AdaptEd`, because its design and much of its code come from `isp_speed_monitor.py` 1.3.0 [21] (see the closed item in §13).

14.1 Third-party components

Nothing below is bundled in the archive. All are installed by the environment file and imported at run time, except the TeX distribution, which is installed separately and run as a program. Licence strings are those declared in each package's own metadata (installed versions shown) or, where noted, on its PyPI or conda-forge page.

Component	Version checked	Declared licence	Role	Combinable with GPL-3.0-or-later
PySide6 / shiboken6	6.12.0	LGPL-3.0-only OR GPL-2.0-only OR GPL-3.0-only	Qt binding	Yes, via the LGPL-3.0 or GPL-3.0 option
Qt.py	2.0.7 (PyPI), 2.0.3 (conda-forge)	MIT	binding abstraction	Yes
pyserial	3.5	BSD	USB virtual COM port	Yes
NumPy	2.5.4	BSD-3-Clause AND 0BSD AND MIT AND Zlib AND CC0-1.0	arrays, ring buffer	Yes
psutil	7.2.2	BSD-3-Clause	RAM readout	Yes
pyqtgraph	0.14.0	MIT	live strip chart	Yes
h5py	3.16.0	BSD-3-Clause	writes the `.vszh5`	Yes
pypdf	6.20.0	BSD-3-Clause	bookmarks in the rendered PDF (optional)	Yes
LuaLaTeX / TeX Live or MiKTeX	TeX Live 2023 (LuaHBTeX 1.17.0) tested	free licences, mainly LPPL 1.3 and GPL (per package)	compiles the session report as a separate program (optional; not installed by the environment file)	Yes: run as an external program, not linked
Veusz	4.2.1 (PyPI and conda-forge pages)	GPL-2.0-or-later	renders PDF/PNG from the `.vszh5` (imported, optional)	Yes, because "or later" admits version 3
CuPy	14.2.0 (PyPI page)	MIT	GPU reductions (optional)	Yes

The right-hand column is my application of the FSF's published compatibility positions, not legal advice; see the confidence table in §12.